

Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

Sumário

1	Introdução à programação orientada a objetos	3
1.1	Um breve histórico	3
1.2	Por que usar orientação a objetos?	4
1.2.1	Reúso	4
1.2.2	Coesão	4
1.2.3	Acoplamento	5
1.2.4	Gap semântico	5
	Referências Bibliográficas	7

1 Introdução à programação orientada a objetos

Toda a base da programação deriva dos algoritmos. Estes consistem em sequências finitas de passos bem definidos, com início e fim claramente delimitados, que buscam a automação de processos. Um algoritmo deve ser suficientemente genérico para que sua implementação seja viável em diferentes linguagens de programação.

A característica fundamental de uma linguagem, além de sua sintaxe, é o seu **paradigma** de programação. Este define a abordagem sob a qual o algoritmo deve ser estruturado para que seus passos alcancem os objetivos propostos Leite et al. (2016).

O paradigma mais utilizado para introduzir a lógica de programação é o **estruturado/procedural**. Por ser um dos modelos mais simples e intuitivos para a expressão de algoritmos, ele frequentemente permite uma transição direta entre a representação lógica e a implementação na linguagem. As linguagens mais conhecidas que adotam esse paradigma são C e Pascal Leite et al. (2016).

Com o avanço do tempo e a crescente demanda por sistemas complexos, o paradigma estruturado tornou-se insuficiente para as novas exigências da engenharia de software. A necessidade de abstrações que aproximassem a tecnologia das entidades do cotidiano impulsionou a criação e o desenvolvimento do Paradigma Orientado a Objetos.

1.1 Um breve histórico

A origem da programação e do paradigma orientado a objetos vem da Universidade de Oslo na Noruega na década de 60. Os pesquisadores Kristen e Ole desenvolveram a primeira linguagem orientada a objetos a “Simula” com objetivo de simulações de eventos discretos (daí seu nome).

Apesar da Simula ser a pioneira, ela foi desenvolvida para uma máquina (UNIVAC 1107) o que a tornava pouco portátil. entretanto na década de 70, Alan Kay desenvolveu uma linguagem orientada a objeto que se tornaria uma das mais conhecidas, a Smalltalk-80.

Muitas linguagens surgiram depois de Smalltalk-80, como C++, Java, C#, Python e etc. E apesar de não ser muito utilizada nos dias atuais, a Smalltalk serviu de inspiração para muitas linguagens orientadas a objetos (O.O).

1.2 Por que usar orientação a objetos?

Além da programação estruturada não representar as informações próximo ao mundo real, ela foca em dados e operações desassociadas. Ou seja, diversos conceitos ficam misturados, logo, não fica claro qual operação está ligada a um determinado dado Leite et al. (2016).

A orientação a objetos foca na organização e interação dos objetos entre si. A Figura 1.1 mostra a diferença entre os paradigmas.

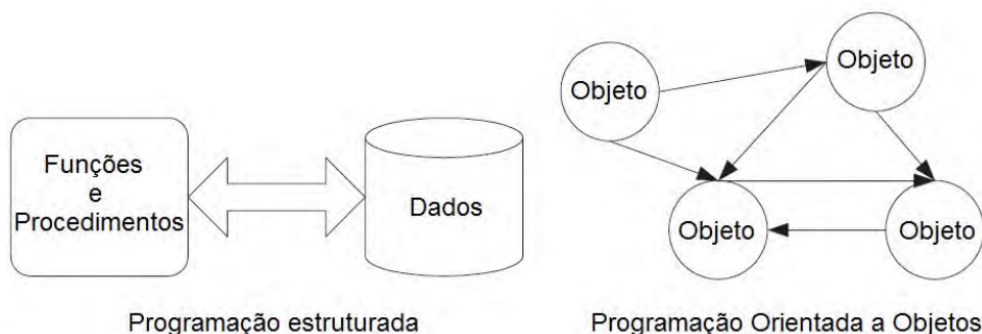


Figura 1.1: Comparativos entre paradigmas de programação.

Fonte: Leite et al. (2016)

Outros fatores para o uso de O.O. descritos por Leite et al. (2016) estão nas subseções a seguir.

1.2.1 Reúso

O reaproveitamento em linguagens estruturadas é ineficiente e propenso a falhas, muitas vezes dependente de cópias manuais ou de sistemas de módulos pouco integrados. A Orientação a Objetos supera isso ao utilizar mecanismos como a herança e a associação, que permitem estabelecer conexões estruturais entre componentes, tornando o reuso prático, seguro e mais próximo das necessidades reais do desenvolvimento de software (Leite et al. (2016)).

1.2.2 Coesão

A organização interna do sistema é um ponto crítico. Em modelos estruturados, é comum que funções ou módulos acumulem múltiplas responsabilidades, criando blocos de código densos e difíceis de manter.

O paradigma orientado a objetos propõe a criação de classes que concentram cada tarefa em uma unidade específica e bem definida. Esse foco em uma única responsabilidade facilita a leitura, a manutenção e a expansão do sistema sem gerar efeitos colaterais indesejados (Leite et al. (2016)).

1.2.3 Acoplamento

O excesso de dependências entre diferentes partes de um software gera fragilidade, onde alterações em um ponto podem interromper funcionalidades em outro.

A Orientação a Objetos introduz estruturas que possibilitam um acoplamento mais flexível, permitindo que as classes interajam de forma controlada. Isso resulta em sistemas mais independentes, nos quais os componentes podem evoluir sem comprometer a integridade de todo o conjunto (Leite et al. (2016)).

1.2.4 Gap semântico

Programar em modelos estruturados exige que o desenvolvedor foque excessivamente na lógica de execução técnica da máquina, distanciando o código dos conceitos do cotidiano.

A Orientação a Objetos atua como uma ponte, oferecendo ferramentas para que a estrutura do programa reflita diretamente os termos, os processos e a lógica do domínio do problema. Essa aproximação reduz o esforço mental necessário para traduzir desafios reais em soluções computacionais eficientes (Leite et al. (2016)).

A tabela 1.1 apresenta um comparativo entre os paradigmas estruturado e orientado a objetos.

Tabela 1.1: Comparativo Avançado: Programação Estruturada vs. Orientação a Objetos

Critério	Programação Estruturada	Orientação a Objetos
Foco Principal	Focado em funções, procedimentos e sequências de passos lógicos.	Focado na organização e interação direta entre objetos.
Visão de Dados	Dados e operações encontram-se separados e desassociados.	Dados (atributos) e comportamentos (métodos) encapsulados em uma única unidade (classe).
Reúso	Ineficiente e manual; propenso a falhas.	Prático e seguro via herança e associação.
Coesão	Módulos e funções acumulam múltiplas responsabilidades.	Classes focadas em uma única responsabilidade.
Acoplamento	Excesso de dependências rígidas e frágeis entre as partes.	Flexível e controlado, garantindo independência aos componentes.
Gap Semântico	Alto; focado excessivamente na lógica técnica da máquina.	Baixo; a estrutura do código reflete o domínio do problema.
Extensibilidade	Mais complexa e custosa em grandes sistemas devido ao acoplamento.	Alta, permitindo estender funcionalidades de forma isolada via herança e polimorfismo.

No próximo capítulo serão abordados os primeiros conceitos de orientação a objetos.